# Annotation Schema: Developer Intent in Code Review Comments

## Project Overview

This schema governs the manual annotation of code review comments for the study **"Understanding Developer Intent in Code Review Comments: A Comparative Study of NLP Models and Contextual Information."** The goal is to classify each comment by the communicative intent the author pursued, enabling downstream comparison of NLP models under varying contextual conditions.

**Annotation task.** Two annotators independently assign one intent label to each review comment, using a standardized set of categories defined in this document. Disagreements are resolved through discussion and, when necessary, adjudication by a third judge.

**Pre-comment context provided to both annotators for every item:**

- The full review comment text
- The code diff or file excerpt the comment references
- The surrounding review discussion thread (prior and subsequent comments)

Annotators must read all three context sources before labeling. The label reflects what the comment *does* in context, not what its surface form suggests in isolation.

## Intent Taxonomy

Eight mutually distinguishable intent categories are defined below. Each carries a two-letter code used on the annotation form.

| Code | Label | Core question answered by the label |
|---|---|---|
| RC | Request a Change | Does the author direct the recipient to modify something? |
| SP | Suggest / Propose | Does the author offer an alternative without requiring action? |
| RQ | Request Clarification | Does the author seek information, rationale, or understanding? |
| CN | Raise a Concern/Uncertainty | Does the author flag a potential problem with no prescribed fix? |

| | | |
|---|---|---|
| EX | Provide an Explanation | Does the author supply rationale, information, or an answer? |
| AK | Acknowledge | Does the author express a stance (approval, disagreement, appreciation)? |
| CR | Coordinate the Review | Does the author manage workflow, timing, assignment, or follow-up? |
| VF | Request Validation | Does the author ask the recipient to empirically verify behavior (by running tests, reproducing a bug, or checking a platform)? |
| OB | Report an Observation | Does the author state a factual finding about existing code without directing action or evaluating it as a problem? |
| AT | Action Taken | Does the author report that a change, fix, or investigation has already been completed? |

---

## RC: Request a Change

**Definition.** The comment explicitly or implicitly directs the recipient to modify code, documentation, a test, a configuration, or another artifact. The author's expectation is that an action will be taken.

**Markers.** Imperative verbs (*rename*, *remove*, *add*, *fix*, *update*); modal obligations (*should*, *must*, *needs to*, *have to*); future-obligation phrasing (*this will need to be changed*).

**Prototypical examples.**

- *"Please rename `data` to `userData` for clarity."*
- *"This function should return early if the list is empty."*
- *"Remove the TODO comment before merging."*

**Boundary notes.** RC requires a clear expectation of recipient action. A comment that merely identifies a problem without specifying or implying what should be done is CN. A comment that offers an alternative as one option among many, without obligation, is SP.

---

## SP : Suggest / Propose

**Definition.** The author offers an alternative approach, implementation, or phrasing as a possibility, not a requirement. The recipient may accept or decline without necessarily reopening discussion.

**Markers.** Hedged language (*you could*, *one option would be*, *consider*, *might want to*, *alternatively*, *what about*); explicit optionality (*just a thought*, *up to you*, *not a blocker*).

**Prototypical examples.**

- *"You could also use `Optional.ofNullable` here to avoid the null check."*
- *"Consider extracting this into a helper method : makes testing easier, but fine as-is too."*
- *"Alternatively, `Map.getOrDefault` would be a bit more concise."*

**Boundary notes.** The key contrast with RC is the author's stance on obligation. If the comment frames an alternative as obligatory (*"you should use…"*) it is RC; if it frames it as optional (*"you could use…"*) it is SP. When in doubt, ask: would the reviewer reopen discussion if the author ignored this? If yes → RC. If likely no → SP.

---

## RQ : Request Clarification

**Definition.** The author seeks information, understanding, justification, or confirmation from the recipient. The comment is oriented toward receiving a response rather than prompting a code change.

**Markers.** Direct questions (*Why did you…? What does this…? Is this…?*); embedded questions (*I'm wondering whether…*); request for explanation (*Can you explain…? Walk me through…*).

**Prototypical examples.**

- *"Why did you choose a HashMap here instead of a TreeMap?"*
- *"What's the expected behavior when `count` is zero?"*
- *"Is there a test covering the edge case where the list is null?"*

**Boundary notes.** An RQ that implies a problem (e.g., *"Why is this not null-checked?"*) may co-occur with CN. Assign RQ as the primary label; add CN only if the concern is independently expressible (see Section 6). Rhetorical questions that make a directive (*"Shouldn't this be snake_case?"*) are RC, not RQ.

---

## CN : Raise a Concern/Uncertainty

**Definition.** The author reports a potential problem, risk, or issue without specifying what action should be taken or even whether action is required. The comment invites discussion or draws attention.

**Markers.** Hedged problem statements (*this might*, *I think there could be*, *I'm not sure this handles*); worry expressions (*I'm concerned that*, *worried about*, *this looks like it could*).

**Prototypical examples.**

- *"This might cause a race condition if two threads hit this path simultaneously."*
- *"I'm not sure this handles the case where the response is empty."*
- *"This could be a performance bottleneck for large payloads."*

**Boundary notes.** CN differs from RC in that no fix is requested or implied. If the author both identifies a problem and tells the recipient what to do, label RC (the directive subsumes the concern). CN and RQ frequently co-occur: a question that identifies a risk (*"Have we handled the null case here?"*) should receive both labels.

---

## EX : Provide an Explanation

**Definition.** The author supplies rationale, context, additional information, or a direct answer to a prior question. The comment is informational and recipient-oriented.

**Markers.** Causal connectors (*because*, *this is because*, *the reason is*); contextual setters (*for context*, *to clarify*, *note that*); direct answers (*yes, that's intentional*, *this handles…*).

**Prototypical examples.**

- *"I used a HashMap here because lookup is O(1) and we expect thousands of entries."*
- *"This is intentional : the legacy API requires this field even when empty."*
- *"The test is skipped because this service isn't yet deployed in CI."*

**Boundary notes.** EX is the primary label for responses to RQ comments. It may co-occur with AK when the author simultaneously provides information and acknowledges the reviewer's point. Distinguish EX from AK: EX adds informational content; AK registers a stance without adding new content.

---

## AK : Acknowledge

**Definition.** The author expresses a stance toward the reviewer's comment or the review as a whole : approval, disagreement, appreciation, or acceptance : without necessarily adding new information.

**Markers.** Approval: *"LGTM"*, *"Good catch!"*, *"Looks good"*, *"Makes sense"*. Disagreement: *"I disagree"*, *"I don't think"*, *"Actually"*. Neutral receipt: *"Noted"*, *"Done"*, *"Fixed"*, *"Will address"*.

**Boundary notes.** AK differs from EX in that it does not convey substantive new information : it registers a relational or evaluative stance. A comment such as *"Good catch, I'll fix it"* should be **RC** : the directive is the operative intent. Sub-types are recorded in a secondary field only.

---

## CR : Coordinate the Review

**Definition.** The author manages review logistics: assignments, timing, blocking status, escalation, or follow-up actions unrelated to the code itself.

**Markers.** Reviewer assignments (*assigning to @…*); timing signals (*I'll look at this after…*, *blocking on…*); status updates (*marking as resolved*, *re-requesting review*).

**Prototypical examples.**

- *"Assigning to @alice since this touches the auth module."*
- *"I'll revisit after the DB migration lands next week."*
- *"Marking this thread resolved : addressed in commit abc123."*

**Boundary notes.** CR is process-oriented; it does not evaluate or change code. A comment that both coordinates and makes a judgment (*"Blocking this PR until the null check is added"*) should receive **RC** : the directive governs.

---

## VF: Request Validation

**Definition.** The author asks the recipient to empirically confirm behavior :by running tests, verifying on a specific platform, reproducing a bug, or providing evidence : rather than merely explaining it.

**Markers.** Verification requests (*"Can you verify…"*, *"Does this work on…"*, *"Could you confirm…"*); test-addition requests (*"Can you add a test for…"*); reproduction requests (*"Can you reproduce…"*).

**Prototypical examples.**

- "Can you verify this works on Windows before we merge?"
- "Can you add a test that reproduces this edge case?"
- "Could you confirm the behavior when the list is empty?"

**Boundary notes.** VF vs RQ: Both categories use interrogative form, but VF demands an empirical action (run code, test a scenario, reproduce a bug) while RQ seeks a conceptual or informational answer. If the expected reply is running or testing something, assign VF; if explaining something, assign RQ. VF vs RC: RC directs a code *change*; VF directs a verification *action* on existing code. "Fix this edge case" → RC. "Can you verify this edge case is handled?" → VF.

---

## OB : Report an Observation

**Definition.** The author states a factual finding about existing code, prior behavior, or a detected pattern : without directing action or evaluating the finding as a problem.

**Markers.** Discovery language (*"The existing implementation already…"*, *"I noticed that…"*, *"This already handles…"*); descriptive present tense (*"Currently, this…"*, *"Looking at the diff…"*).

**Prototypical examples.**

- "The existing implementation already handles this case."
- "I noticed this method is called in three other places."
- "This was added in the refactor last month."

**Boundary notes.** OB vs EX: EX provides reasoning to explain *why* something is as it is; OB states *what* is. "The cache is invalidated on logout" → OB. "The cache is invalidated on logout *because* the session token changes" → EX. OB vs CN: If the observation implies a defect or risk, assign CN. If the observation is neutral or informative : even favorable : assign OB.

---

## AT : Action Taken

**Definition.** The author reports that a change, fix, investigation, or other action has already been completed : providing a status update on work that is done, not planned.

**Markers.** Completion statements (*"I've updated…"*, *"Fixed in commit…"*, *"Pushed a fix…"*, *"Added a test…"*); investigation reports (*"I looked into this : it turns out…"*); past-tense action descriptions.

**Prototypical examples.**

- "I've updated the tests and pushed the fix."
- "Fixed in commit a3b4c : the null check was missing."
- "I looked into this; it's expected behavior per the spec."

**Boundary notes.** AT vs CR: CR coordinates *future* workflow (what will happen); AT reports *completed* work (what has happened). "I'll address this before merging" → CR. "Addressed in the latest commit" → AT. AT vs AK: AK expresses a *stance* (appreciation, agreement); AT reports an *action*. "Good catch!" → AK. "I've addressed this in the latest push" → AT.

# Annotation Guidelines

## Step-by-Step Annotation Process

1. **Read the full context first.** Before forming any judgment, read: (a) the diff or code excerpt, (b) the review discussion thread from top to bottom, (c) the target comment. Do not begin labeling at step (c).
2. **Identify what the comment does, not what it says.** Focus on the communicative act performed in context. A question that functions as a directive should be labeled RC, not RQ.
3. **Assign primary label.** Apply one label for which the comment clearly performs a distinct act.
4. **Rate your confidence.** Use the three-point scale defined in Section 6.
5. **Record a rationale for any non-obvious label.** One sentence is sufficient. This field is mandatory when confidence is Low or when more than two labels are assigned.
6. **Flag items for discussion.** Use the flag field for any item you believe should be discussed at the reconciliation meeting (even if you have labeled it).

---

## Using Pre-Comment Context

**The code diff or file excerpt** establishes what artifact is being reviewed. It answers whether a concern about correctness or performance is plausible given the actual code.

**The review discussion thread** establishes:

- Whether an earlier comment already requested the same change (affects whether a new comment is RC or AK signaling compliance)
- Whether the comment is a reply (affects whether EX applies)
- Whether the author is the PR owner or an external reviewer (matters for CR and AK)

**What context cannot override.** Context informs but does not rewrite meaning. If a comment's text is clearly a directive, context cannot make it RQ. If the surrounding thread makes a question's answer obvious, the question is still RQ (unless it is clearly rhetorical).

---

## Labeling Don'ts

- **Do not label based on length.** Short comments are not necessarily NS; long comments are not necessarily EX.
- **Do not infer unstated intent.** If a comment does not raise a concern, do not assign CN because you personally see a risk in the code.

- **Do not conflate the topic with intent.** A comment about security is not automatically CN; it may be EX (explaining a security decision) or RC (requiring a security fix).
- **Do not assign AK to every reply.** A reply that only provides information without expressing a stance is EX, not AK.
- **Do not use NS as a residual category.** NS requires that the issue be cosmetic or trivially minor. When uncertain, prefer RC or SP over NS.

---

## Annotation Form Fields

| Field | Required | Values / Format |
|---|---|---|
| Comment ID | Yes | Provided by the task manager |
| Primary label | Yes | One or more of: RC, SP, RQ, CN, EX, AK, CR, NS |
| Confidence | Yes | High / Medium / Low |

---

## Annotator Independence

Annotators work independently until the reconciliation meeting. They must not:

- Discuss specific items with the other annotator before recording their labels
- Review the other annotator's labels before completing their own
- Consult sources outside this schema document during annotation

Annotators *may* contact the study coordinator with questions of fact (e.g., a term in the code they do not recognize) but must not request label guidance on specific items.

# Single-Label Policy and Confidence Ratings

## Single-Label Assignment

Each comment receives exactly one label — the one that best captures its primary communicative act. When a comment appears to perform more than one act, assign the label for the most operative intent, using the priority order below as the tiebreaker.

**When intents compete.**

- Identify which act the recipient would prioritize — what they would do in response — and assign that label alone (e.g., *"Good catch [AK]. Please fix before merging [RC]."*)
- **A comment that praises and directs is **RC**; one that questions and implies a concern is **CN**; one that coordinates and expresses thanks is **CR****
- **When still uncertain after applying the priority order, mark confidence Low and note the competing labels in the rationale field.**

**Tiebreaker rule.**

- **When two labels are equally plausible, the priority order below decides. The lower-numbered label wins**.
- Never assign multiple labels as a hedge against uncertainty. Pick the best-fitting label and record confidence as Low.
- **The single-label rule is absolute: even a comment that clearly does two things receives one label only**.

**One label per item, always. This rule has no exceptions within a single annotation pas**s.

---

## Priority Order for Tiebreaking

When two labels are both plausible, assign the one with the lower number. The order reflects the act most likely to drive the recipient's response:

1. RC (most action-oriented)
2. VF
3. SP
4. CN
5. RQ
6. EX
7. OB
8. AK
9. AT
10. CR (least consequential / cosmetic)

Example: a comment that praises and directs receives **RC** (priority 1), not AK (priority 8).

---

## Confidence Ratings

Annotators record their confidence in the assigned label using a three-point scale.

| Level | Meaning | Typical triggers |
|---|---|---|
| **High** | The label is clearly correct; a reasonable annotator would almost certainly agree | Unambiguous directive, clear question, explicit nit marker |
| **Medium** | The label is likely correct but a reasonable annotator might choose differently | Hedged language, missing prior context, dual-purpose sentences |
| **Low** | The label is uncertain; the comment genuinely sits between two or more categories | Ambiguous modality, highly implicit intent, very short comment with no context |

**Mandatory rationale.** Annotators must provide a written rationale for any item rated Low confidence . Rationale is recorded in the free-text field on the annotation form.

**Confidence and IAA.** Low-confidence items from both annotators are prioritized in the adjudication phase. A High-confidence disagreement (both annotators confident but assigning different labels) escalated to the third judge immediately.